



IIC2343 - Arquitectura de Computadores (I/2026)

Guía de ejercicios: I/O

Ayudantes: Alberto Maturana (alberto.maturana@uc.cl), Mario Rojas
(mario.denzel@estudiante.uc.cl), Ignacio Gajardo (gajardo.ignacio@uc.cl)

Pregunta 1: Preguntas conceptuales

- (a) En un controlador de interrupciones, ¿qué representa cada bit del *Interrupt Request Register*, del *In-Service Register* y del *Interrupt Mask Register*? Explique la función de cada uno.
- (b) ¿Por qué un controlador DMA genera una interrupción al finalizar una transferencia de datos?
- (c) ¿Qué ventaja presenta el uso de interrupciones frente a *polling* cuando los eventos de un dispositivo I/O son poco frecuentes?
- (d) ¿Por qué se utiliza un vector de interrupciones para almacenar las direcciones de las ISR en lugar de almacenar directamente las rutinas? Mencione dos ventajas de este diseño.
- (e) **(I2 2025-1)** Suponga que, al computador básico estudiado en clases, se le añade todo el soporte en *hardware* necesario para comunicarse con dispositivos I/O mapeados en memoria a través de interrupciones. Además, uno de los dispositivos mapeados corresponde a un *pendrive*, del cual se desea copiar información a la RAM. ¿Es estrictamente necesario incluir *hardware* adicional para cumplir con esta funcionalidad? Justifique su respuesta.
- (f) **(I2 2024-1)** Indique cómo se lleva a cabo la transferencia de datos entre dispositivos I/O y la memoria RAM en arquitecturas que **carecen** de un controlador de DMA.

Solución:

- a) En los tres registros, cada bit está asociado a una *Interrupt Request* (IRQ) distinta. La diferencia entre ellos consiste en la información que almacenan sobre cada IRQ.

En el *Interrupt Request Register* se registra si una IRQ se encuentra pendiente de atención o no. Por otro lado, el *In-Service Register* indica si una IRQ está siendo atendida actualmente. Finalmente, el *Interrupt Mask Register* señala si una IRQ se encuentra enmascarada y, por tanto, debe ser ignorada aunque exista una solicitud pendiente para ellas.

- b) Porque durante la transferencia la CPU no participa directamente en el movimiento de los datos. Al terminar la operación, el DMA debe informar a la CPU que la transferencia terminó para que esta pueda continuar con las acciones dependientes de dichos datos.
- c) Cuando los eventos son poco frecuentes, el *polling* provoca múltiples consultas al estado del dispositivo que, en la mayoría de los casos, no producen ningún cambio relevante, desperdiciando tiempo de procesamiento de la CPU. Las interrupciones evitan este costo, ya que el dispositivo notifica a la CPU únicamente cuando requiere atención.
- d) Una ventaja es que permite que las ISR tengan longitudes variables sin dificultar su búsqueda en memoria, ya que el vector almacena únicamente la dirección de inicio de cada rutina. Otra ventaja corresponde a que facilita la actualización de una ISR, pues basta con modificar la dirección almacenada en la entrada correspondiente del vector a la dirección de la nueva versión de la ISR sin alterar el mecanismo utilizado por la CPU para encontrarla.
- e) En estricto rigor no es necesario. Si bien lo óptimo es contar con un controlador de DMA (*Direct Memory Access*) para orquestrar la transferencia entre dispositivos y la memoria RAM, en la práctica se puede hacer uso de *Programmed I/O* para llevarla a cabo, *i.e.* usar los registros de CPU como puente para transferir los datos. Si bien esto en la práctica no se realiza por la cantidad de ciclos de CPU perdidos, se puede implementar como alternativa en caso de querer un *hardware* más sencillo.
- f) En este caso se lleva a cabo *Programmed I/O*; *i.e.* la transferencia manual de los datos hacia los registros de la CPU, los que luego son transferidos a destino (ya sea la RAM o dispositivos I/O de almacenamiento).

Pregunta 2: Comunicación con dispositivos I/O (I2 2025-2)

Suponga que decide automatizar la disponibilidad de un baño mediante sensores y un emisor Bluetooth conectados a un computador con arquitectura RISC-V. Para la comunicación con los sensores y el emisor, sus registros de 32 bits están mapeados en memoria desde la dirección 0x600DD0D0:

Offset	Nombre	Descripción
0x00	door_sensor_status	Registro estado sensor de puerta
0x04	light_sensor_status	Registro estado sensor de luz
0x08	notifier_command	Registro comando emisor
0x0C	notifier_status	Registro estado emisor

Nombre	Descripción	Valor
door_sensor_status	Puerta cerrada	0
door_sensor_status	Puerta abierta	1
light_sensor_status	Luz apagada	0
light_sensor_status	Luz encendida	1
notifier_status	Inactivo	0
notifier_status	Activo	1
notifier_status	Enviando	2
notifier_command	Encender emisor	1
notifier_command	Notificar OPEN	2
notifier_command	Notificar CLOSED	3
notifier_command	Apagar emisor	4

El sistema debe enviar una notificación Bluetooth según la disponibilidad del baño: OPEN si es que está disponible, y CLOSED si es que está ocupado según los estados de los sensores de puerta (door_sensor_status) y luz (light_sensor_status). Para ello, se desarrolla el siguiente programa:

```
notifier_automation:
    # Carga zero + 0x600DD0D0 en t0
    addi t0, zero, 0x600DD0D0
    addi s0, zero, 0
    addi s1, zero, 1
    addi s2, zero, 2
    addi s3, zero, 3
    addi s4, zero, 4

step_one:
    lw t3, 12(t0) # Carga Mem[t0 + 12] en t3
    # Salta a step_two si t3 != s0
    bne t3, s0, step_two
    sw s1, 8(t0) # Carga s1 en Mem[t0 + 8]

step_two:
    lw t1, 0(t0)
    lw t2, 4(t0)
    bne t1, s1, step_three
    bne t2, s0, step_three
    # Salto incondicional a step_four
    jal zero, step_four
```

```
# => Continuación

step_three:
    sw s3, 8(t0)

while_step_three:
    lw t3, 12(t0)
    # Salta a while_step_three si t3 == s2
    beq t3, s2, while_step_three

    sw s4, 8(t0)
    jal zero, step_one

step_four:
    sw s2, 8(t0)

while_step_four:
    lw t3, 12(t0)
    beq t3, s2, while_step_four

    sw s4, 8(t0)
    jal zero, step_one
```

Describa, basándose en las tablas provistas, las tareas que realiza el programa en cada *label* `step_*`. Haga una descripción según los estados y comandos; no una explicación por cada instrucción del programa.

Solución:

- **step_one:** Se revisa el estado del emisor. Si el emisor está inactivo, se enciende y continúa a **step_two**. En otro caso, salta directamente a **step_two**.
- **step_two:** Se revisa el estado del sensor de puerta y el sensor de luz del baño. Si la puerta está cerrada o la luz está encendida, salta a **step_three**. En otro caso, salta directamente a **step_four**.
- **step_three:** A través del emisor, se envía la notificación **CLOSED**. Luego, se espera a que termine el envío y se apaga el emisor. Finalmente, salta de vuelta al **step_one**.
- **step_four:** A través del emisor, se envía la notificación **OPEN**. Luego, se espera a que termine el envío y se apaga el emisor. Finalmente, salta de vuelta al **step_one**.

Pregunta 3: Comunicación con dispositivos I/O (I2 2024-2)

Suponga que decide, con el propósito de ahorrar tiempo, automatizar la preparación de café filtrado a partir de un molinillo y hervidor eléctricos conectados a un computador con ISA RISC-V. Para la comunicación con los electrodomésticos, sus registros de 32 bits son mapeados en memoria desde la dirección 0x600DCAFE:

Offset	Nombre	Descripción
0x00	coffee_grinder_status	Registro estado molinillo
0x04	coffee_grinder_command	Registro comando molinillo
0x08	coffee_grinder_grind_size	Config. grosor molienda
0x0C	coffee_grinder_weight	Config. gramos a moler
0x10	kettle_status	Registro estado hervidor
0x14	kettle_command	Registro comando hervidor
0x18	kettle_temperature	Config. temperatura hervidor (°C)

Nombre	Descripción	Valor
coffee_grinder_status	Apagado	0
coffee_grinder_status	Encendido	1
coffee_grinder_status	Moliendo	2
kettle_status	Apagado	0
kettle_status	Encendido	1
kettle_status	Hirviendo	2
coffee_grinder_command	Encender	1
coffee_grinder_command	Apagar	255
coffee_grinder_command	Moler	128
kettle_command	Encender	1
kettle_command	Apagar	255
kettle_command	Hervir	128
coffee_grinder_grind_size	Fino	1
coffee_grinder_grind_size	Medio	2
coffee_grinder_grind_size	Grueso	3

Pensando en su preparación favorita, desarrolla el siguiente programa:

```
coffee_system:
# Carga 0 + 0x600DCAFE en t0
addi t0, zero, 0x600DCAFE
addi t2, zero, 0
addi t3, zero, 0
addi s1, zero, 1
addi s2, zero, 2
addi s3, zero, 128
addi s4, zero, 255
addi s5, zero, 30
addi s6, zero, 92

step_one:
    lw t1, 0(t0) # Carga Mem[t0 + 0] en t1
    # Salta a continue_step_one si t1 != s2
    bne t1, s2, continue_step_one
    jal ra, step_five # Llamado a step_five
    beq zero, zero, step_three
    continue_step_one:
        beq t1, s1, step_two
        sw s1, 4(t0) # Carga s1 en Mem[t0 + 4]

step_two:
    sw s2, 8(t0)
    sw s5, 12(t0)
    sw s3, 4(t0)
    jal ra, step_five
# Continúa en step_three -->
```

```
step_three:
    lw t1, 16(t0) # Carga Mem[t0 + 16] en t1
    # Salta a continue_step_three si t1 != s2
    bne t1, s2, continue_step_three
    jal ra, step_six
    beq zero, zero, end # Salto incondicional
    continue_step_three:
        beq t1, s1, step_four
        sw s1, 20(t0)

step_four:
    sw s6, 24(t0)
    sw s3, 20(t0)
    jal ra, step_six
    beq zero, zero, end

step_five:
    while_step_five:
        lw t1, 0(t0)
        beq t1, s2, while_step_five
        sw s4, 4(t0)
        jalr zero, 0(ra) # Retorno

step_six:
    while_step_six:
        lw t1, 16(t0)
        beq t1, s2, while_step_six
        sw s4, 20(t0)
        jalr zero, 0(ra)
end: # Término del programa.
```

Describa, basándose en las tablas provistas, las tareas que realiza el programa en cada *label* `step_*`. Haga una descripción según los estados y comandos; no una explicación por cada instrucción del programa.

Solución: A continuación, se describe lo que se realiza en cada **step**:

- **step_one:** Se revisa el estado del molinillo. Si está moliendo, se ejecuta **step_five** y se salta a **step_three**. Si está encendida, se salta a **step_two**. En otro caso, se enciende antes de continuar con **step_two**.
- **step_two:** Se configura el molinillo con una molienda de grosor medio y un total de 30 gramos. Luego, empieza a moler y se ejecuta **step_five** antes de seguir a **step_three**.
- **step_three:** Se revisa el estado del hervidor. Si está hirviendo, se ejecuta **step_six** y se termina el programa. Si está encendida, se salta a **step_four**. En otro caso, se enciende antes de continuar con **step_four**.
- **step_four:** Se configura el hervidor con una temperatura de 92°C. Luego, empieza a hervir y se ejecuta **step_six** antes de terminar el programa.
- **step_five:** Se revisa el estado del molinillo hasta que deje de moler.
- **step_six:** Se revisa el estado del hervidor hasta que deje de hervir.